

CHAPTER 15

User Defined Primitives

Welding



15.1 User Defined PrimitiveS (UDPs)

User Defined PrimitiveS (UDPs) 用來建立 (modeling) 用於 Verilog 電路模擬的電路，它只能用於模擬而已，並不能用於電路合成 (Non-synthesizable)。

UDP 在電路模擬時比 module 來得省時、有效率，通常用來建立 ASIC 模擬用的 cell library。

15.1.1 UDP 的特性

UDP 具有下列幾項特性：

- Verilog 叫用 UDP 的方法和叫用 module 是一樣的。
- 用表格 (table) 這個語法來建立與描述 UDP。
- UDP 一定要有一至十個輸入訊號，它的值可以是 0、1 或 x (不確定值，Unknown Value)；如果硬是給 z (高阻抗，High Impedance)，相當於是給 x。
- UDP 一定有一個而且也只會有一個輸出訊號，它的值只會是 0、1 或 x，您不能將 z 給定 UDP 當輸出值。如果想要有二個輸出訊號就要分開考量設計成二個 UDP，那麼才會有二個輸出值。
- 如果 UDP 的輸入訊號包含 z，那麼 UDP 唯一的輸出訊號必定是 x。
- 如果在表格 (table) 這個語法中沒有將各種輸入訊號的值都考量而描述成的 UDP，當在做電路模擬時剛好有讓輸入訊號變成沒有考量到的值，那麼輸出訊號變成 x (不確定值，Unknown Value)。
- 您建立出來的 UDP 可以是組合邏輯 (Combinational) UDP 或者是循序邏輯 (Sequential) UDP。

15.1.2 可用於 UDP 的邏輯值與 Edge Trigger 值

可用於 UDP 的邏輯值與 Edge Trigger 值，如下表：

符 號	意 義
0	邏輯 0
1	邏輯 1
x	不確定值，Unknown Value
?	0、1 或 x
*	(??) 表示輸出由 ? (0、1 或 x) 變成 (0、1 或 x)
B 或 b	0 或 1
-	輸出值不變
(ab)	輸出由 a 變成 b 例如：(01) 表示輸出由 0 變成 1
R 或 r Rising Edge	(01) 輸出由 0 變成 1
F 或 f Falling Edge	(10) 輸出由 1 變成 0
P 或 p Positive	(01) 輸出由 0 變成 1 (0x) 輸出由 0 變成 x (x1) 輸出由 x 變成 1
N 或 n Negative	(10) 輸出由 1 變成 0 (x0) 輸出由 x 變成 0 (1x) 輸出由 1 變成 x

15.1.3 UDP 的 BNF 表示法

UDP 的 BNF 表示法

```

primitive <UDP_name> (<port_name_list>);
  <port_declaration>;
  table
    <table_definition>
  endtable
endprimitive

```

```
table definition
  <input_composition_1>:<out_value_1>
  <input_composition_2>:<out_value_2>
  ...
endtable
```



15.2 組合邏輯 UDP

組合邏輯 UDP 的輸出是由輸入來決定的，並且它輸出訊號的資料型態只能是 wire。

例 1 2 對 1 的多工器

假設 2 對 1 的多工器的 function，如下表：

a	b	sel	y	說明
0	?	0	0	sel = 0，則 y = a
?	0	1	0	sel = 1，則 y = b
1	?	0	1	sel = 0，則 y = a
?	1	1	1	sel = 1，則 y = b

假設多了下列 function 的考量：

a	b	sel	y	說明
0	0	?	0	只要 a = 0，且 b = 0，就令 y = 0
1	1	?	1	只要 a = 1，且 b = 1，就令 y = 1

【2 對 1 的多工器的 Verilog 程式碼：Mux2to1.v】

```
primitive Mux2to1 (y, a, b, sel); // Combinational UDP
  output y;
  input a, b, sel;
```

```

table
// a b sel : y
0 ? 0 : 0;
? 0 1 : 0;

1 ? 0 : 1;
? 1 1 : 1;

// 0 0 ? : 0; // 只要 a = 0 且 b = 0 就令 y = 0
// 1 1 ? : 1; // 只要 a = 1 且 b = 1 就令 y = 1
endtable
endprimitive

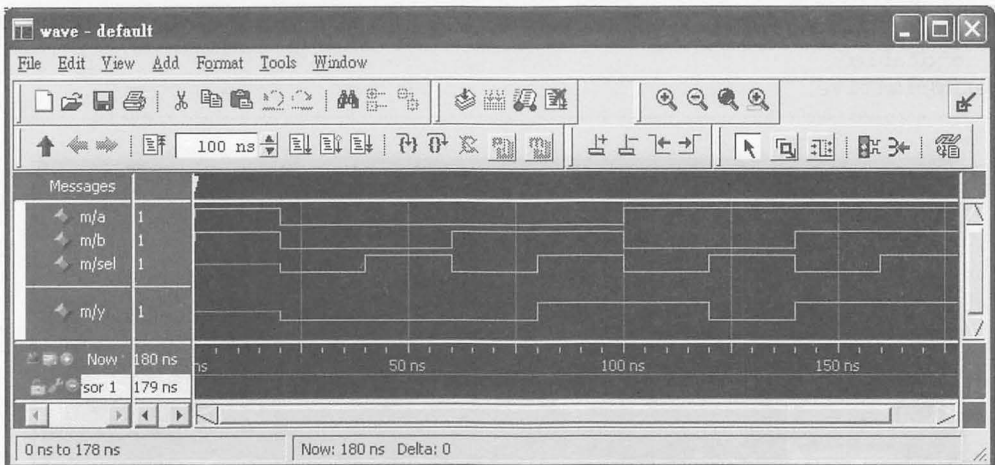
```

【叫用 Mux2to1 的 Verilog 程式碼 : Wrap_Mux2to1.v】

```

module Wrap_Mux2to1(y, a, b, sel);
input  a, b, sel;
output y;
wire  y;
    Mux2to1 (y, a, b, sel);
endmodule

```



▲ Wrap_Mux2to1 裡頭 Mux2to1 的模擬結果

例 2 truth table 真值表輸出考量

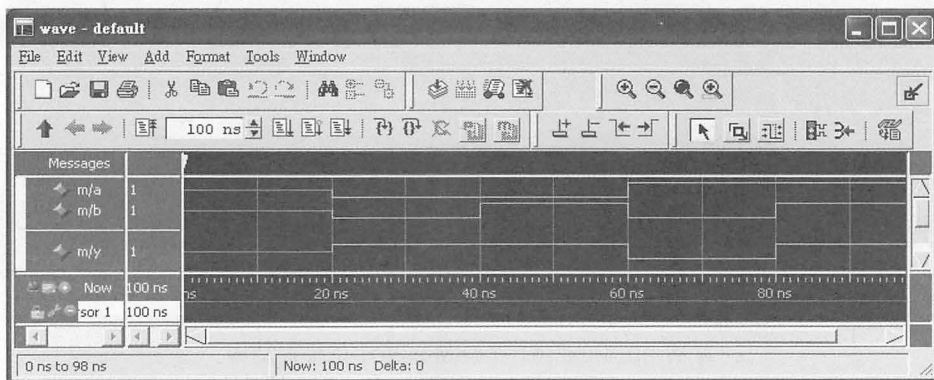
假設您需要對某個 truth table 的輸入來決定輸出值的 function，如下表：

a	b	y	說明
0	?	1	只要 a=0，就令 y=1
1	0	0	a=1，且 b=0，則 y=0
1	x	0	a=1，且 b=x，則 y=0
1	1	1	a=1，且 b=1，則 y=1

【truth table 真值表輸出值考量的 Verilog 程式碼：truth_table.v】

```
primitive truth_table (y, a, b);
    output y;
    input  a, b;

    // Truth table, y(a, b)
    table
    // a b : y
    0 ? : 1;
    1 0 : 0;
    1 x : 0;
    1 1 : 1;
    endtable
endprimitive
```



▲ truth_table 的模擬結果



15.3 循序邏輯 UDP

循序邏輯 UDP 的輸出除了由輸入決定以外還可以和目前 output 訊號 reg 的值一起決定，它輸出訊號的資料型態只能是 reg，而且可以給定初始值，這樣的話電路一開始模擬時就不會是 x (不確定值，Unknown Value)。

循序邏輯 UDP 可以是準位觸發 (Level Trigger) 與正/負緣觸發 (Edge Trigger)。

循序邏輯 UDP 若是正/負緣觸發 (Edge Trigger)，那麼在 UDP 表格描述 (table) 這個語法中必定有一行的輸入訊號值是 r、R、f、F、p、P、n 或 N 這一類的 Edge Trigger，而且在這一行之中只能有一個 Edge Trigger。

例 1 D 型 Latch

用 UDP 設計 D 型 Latch 的真值表，如下表：

En	D	目前 Q	Q	說明
1	0	?	0	只要 En = 1，且 D = 0，就令 Q = 0
1	1	?	1	只要 En = 1，且 D = 1，就令 Q = 1
0	?	?	-	只要 En = 0，就令 Q 的值保持不變
?	0	0	0	只要 D = 0，且目前 Q = 0，就令 Q = 0
?	1	1	1	只要 D = 1，且目前 Q = 1，就令 Q = 1

【D 型 Latch 的 Verilog 程式碼：Latch.v】

```
primitive Latch (Q, En, D);
  output Q;
  reg    Q; // Level sensitive UDP
  input  En, D;

  table
  // En D : Q' : Q
```

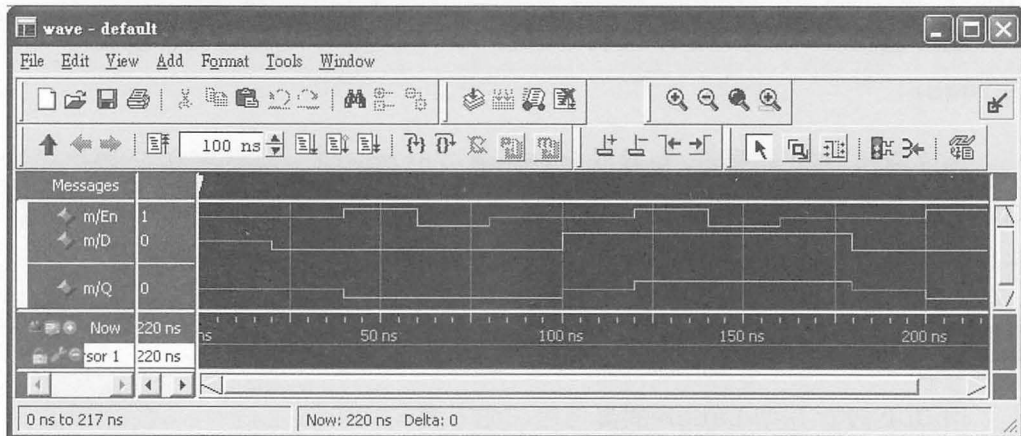


```

1 0 : ? : 0;
1 1 : ? : 1;
0 ? : ? : -; // 保值目前的值
? 0 : 0 : 0;
? 1 : 1 : 1;

endtable
endprimitive

```



▲ D 型 Latch 的模擬結果

例 2 D 型 Flip-Flop

用 UDP 設計 D 型 Flip-Flop 的真值表，如下表：

Clk	D	目前 Q	Q	說明
0→1	0	?	0	Clk 由 0→1，且 D=0，就令 Q=0
0→1	1	?	1	Clk 由 0→1，且 D=1，就令 Q=1
1→0	?	?	-	Clk 由 1→0，就令 Q 的值保持不變

【D 型 Flip-Flop 的 Verilog 程式碼：DFF.v】

```

primitive DFF (Q, Clk, D);
output Q;
reg Q; // Edge sensitive UDP

```



```

input    Clk, D;

initial  Q = 1'b1;

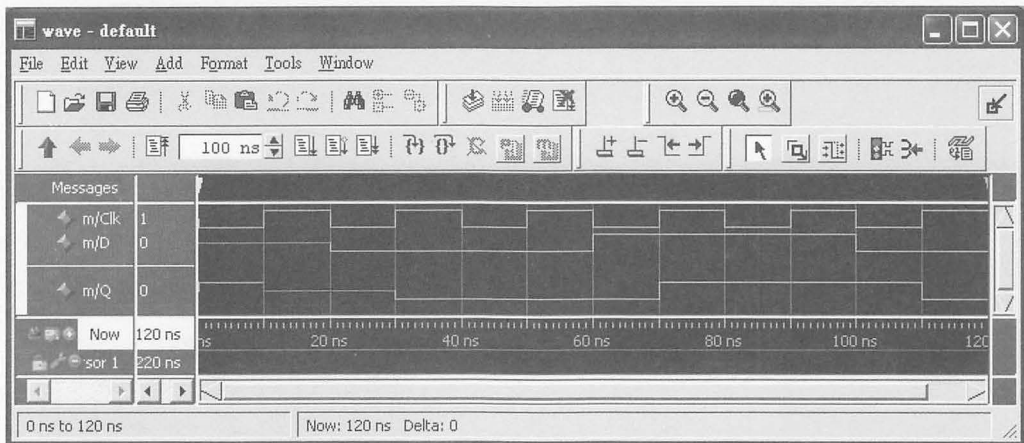
table
  // Clk   D       : Q'   : Q
    r      0       : ?    : 0; // Rising Clock, D = 0
    r      1       : ?    : 1; // Rising Clock, D = 1
    (0x) 0       : 0    : 0; // Rising Clock, D = 0
    (0x) 1       : 1    : 1; // Rising Clock, D = 1

    f      ?       : ?    : -; // Ignore falling Clock
    (x0) ?       : ?    : -; // Ignore falling Clock

    (?1) 0       : 0    : -; // Possible Clock, output unchanged
    (?1) 1       : 1    : -; // Possible Clock, output unchanged

    ?      (??)    : ?    : -; // Ignore data change on steady Clock
endtable
endprimitive

```



▲ D 型 Flip-Flop 的模擬結果

本章練習

Question 1 :

什麼時候會用到 Verilog 的 User Defined Primitives ?

Question 2 :

組合邏輯 UDP 與循序邏輯 UDP 的使用時機各是 ?

PS Google 可以幫您找到解答。